Journal of Software: Evolution and Process

WILEY

Software: Evolution and Process

RESEARCH ARTICLE - EMPIRICAL OPEN ACCESS

Engineering MLOps Pipelines With Data Quality: A Case Study on Tabular Datasets in Kaggle

Matteo Pancini[1] | Matteo Camilli[1] | Giovanni Quattrocchi[1] | Damian Andrew Tamburri[2,3]

[1]Department of Electronics, Information and Bioengineering (DEIB), Politecnico di Milano, Milan, Italy | [2]Department of Engineering, University of Sannio, Benevento, Italy | [3]NXP Semiconductors N.V. (NXP), Eindhoven, the Netherlands

Correspondence: Giovanni Quattrocchi (giovanni.quattrocchi@polimi.it)

Received: 24 April 2024 | **Revised:** 8 April 2025 | **Accepted:** 29 July 2025

Keywords: data quality | Kaggle | machine learning | MLOps

ABSTRACT

Ensuring high-quality data is crucial for the successful deployment of machine learning models, thereby sustaining the operational pipelines around such models. However, a significant number of practitioners do not currently use data quality checks or measurements as gateways for their model construction and operationalization, indicating a need for greater awareness and adoption of these tools. In this study, we propose an automated approach for automating the process of architecting machine learning pipelines by means of (semi-)automated data quality checks. We focus on tabular data as a representative of the most widely used structured data formats in said pipelines. Our work is based on a subset of metrics that are particularly relevant in MLOps pipelines, stemming from our engagement with expert practitioners in machine learning operations (MLOps). We selected Deepchecks, a well-known tool for conducting data quality checks, from a cohort of similar tools to evaluate the quality of datasets collected from Kaggle, a widely used platform for machine learning competitions and data science projects. We also analyze the main features used by Kaggle to rank their datasets and used these features to validate the relevance of our approach. Our approach shows the potential for automated data quality checks to improve the efficiency and effectiveness of MLOps pipelines and their operation, by decreasing the risk of introducing errors and biases into machine learning models in production.

1 | Introduction

Artificial intelligence (AI) is transforming all industries and technologies [1] present in our production and societal tissue. At the same time, the lack of training or knowledge of software engineering processes and best practices [2] often compromises the quality—and even simply the deployability—of many AI applications, especially wherefore those applications reach the stage in which machine learning operations (MLOps) are needed to operationalize and expose such models to their intended users [3]. This is true for all machine learning (ML) applications within which developers have limited knowledge of software quality practices or processes [4]. In particular, as noted by Jain et al. [5], practitioners often overlook the importance of data quality in their workflows, focusing primarily on model performance while neglecting the impact of poor-quality data on overall outcomes. This lack of attention to data quality checks can lead to biases, errors, and inefficiencies during model construction and operationalization, emphasizing the need for greater awareness and adoption of data quality tools with a specific focus to software engineering, maintenance and evolution included.

In this paper, we examine the importance of data quality and the main metrics to analyze in ML tasks, exerting the specific focus above. Specifically, we evaluate the current state of data quality software tools and investigate the most pertinent metrics in the field of ML with the general goal of incepting automation capable of improving the correctness by construction of MLOps pipelines by fashioning metrics and quality gateways as is typical in

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2025 The Author(s). Journal of Software: Evolution and Process published by John Wiley & Sons Ltd.

more classical DevOps pipelines [6]. By doing so, we aim to shed light on how data quality can be ensured in MLOps pipelines, which are becoming increasingly important in almost all fields of human knowledge processing.

To attain our results, we analyze Kaggle repositories for classification tasks on tabular datasets to provide practical insights into data quality issues they present. Kaggle is a popular platform for ML professionals and enthusiasts, offering a repository of datasets and notebooks.[1] By examining these repositories through conventional techniques inherited from the field Mining Software Repositories [7], we obtain a better understanding of the design challenges that practitioners of ML operations face when working with such datasets. We emphasize the fundamental data quality issues with these datasets in particular, as well as the significance of data analysis and pre-processing.

Our analysis concludes that identifying specific data quality issues leads to defining design principles that MLOps practitioners shall enforce in their own catered data pipelines, most commonly (a) missing data, (b) feature inconsistencies, and (c) unknown data biases. In addition, we evaluate the efficacy of existing data quality software tools as gateways in the aforementioned MLOps pipelines, used in addressing these concerns. By doing so, we clarify the current state of data quality in MLOps pipelines and identify areas where software architecture and other automation enhancements can be made to ensure that ML operations are accurate, trustworthy, and production-ready.

The remainder of the paper is organized as follows. In Section 2, we discuss related work. In Section 3, we introduce the methodology used to guide our research activity as well as the research questions we aim to answer. In Section 4, we present our empirical evaluation. In Section 4.6, we discuss major limitations of the our work. We finally conclude the paper and we present future directions of our research in Section 5.

2 | Related Work

In the last decade, the critical role of data quality in ML has garnered increasing attention from both academia and industry. Jain et al. [5] note that while significant advancements have been made in enhancing ML model quality, the quality of the training data has not received comparable attention. The paper emphasizes that the quality of training data is crucial for model performance.

As described by Zhang et al. [8], data quality is a fundamental concern in ML systems, as the performance of these systems relies heavily on the quality of the data used during training and evaluation. Detecting data bugs early is essential, given that predictions are often logged and used to generate additional data, creating feedback loops that can amplify even minor issues.

For training data, rule-based methods such as Data Linters [9] have been proposed. These methods identify issues like miscoded values, outliers, and organizational errors. Metrics to assess training data coverage have also been introduced to ensure all critical scenarios are addressed. The consistency between training and test data, known as skew, is another critical area of concern [10]. Proposed methods for detecting skew include evaluating the likelihood of training data similarity using kernel density estimation and measuring differences in neuron activation traces. Feature comparisons and distribution-based metrics, such as changes in probability distributions, have also been suggested as intuitive methods for assessing data skew.

Frameworks have been developed to address data quality issues systematically. Data validation systems, such as the one integrated into Google's TFX platform [11], apply constraints to detect data issues and measure discrepancies between training and new data. Traditional data cleaning methods from database and data warehousing domains can also be exploited for ML applications. These approaches, combined with ML-specific tools and frameworks, underscore the importance of rigorous data quality management in the MLOps lifecycle.

Supporting this view, Vanschoren et al. [12] introduced OpenML, an open network for managing ML projects, demonstrating how poor data quality can significantly diminish model accuracy. Further investigations by Krishna et al. [13] have identified specific data quality issues, such as missing data and incorrect labels, that directly impact ML outcomes. Hence, rigorous assessment and improvement of data quality are key for the reliability and accuracy of ML applications.

According to Kaminwar et al. [14], data quality introduces a separate class of requirements emphasizing diversity, completeness, consistency, and correctness of the data. Completeness ensures sufficient representation across the range of expected data values, while consistency mandates uniform data formatting. Correctness refers to the authenticity of the data, which must be validated to prevent adversarial manipulation. These properties are critical for effective validation techniques, including cross-validation, as the reliability of such techniques depends heavily on the underlying data quality.

Additionally, studies like Farias et al. [15] illustrate the utility of mining software repositories. Such practices enable the extraction of valuable data that aids in understanding development processes, detecting patterns, forecasting trends, and improving quality. Similarly, Bosu et al. [16] highlight the importance of data quality in empirical software engineering, particularly noting its relevance for tasks such as software effort estimation and defect prediction. Finally, Wu et al. [17] explore the prediction of security bug reports, emphasizing the need for high-quality data for accurate forecasts.

While significant advancements have been made in addressing data quality challenges in ML, there is a lack of empirical studies investigating how practitioners navigate data quality issues in applied settings, such as those encountered in ML pipelines. We address this gap by providing an empirical analysis of Kaggle repositories, specifically focusing on classification tasks using tabular datasets. By leveraging techniques from mining software repositories, we uncover practical insights into the fundamental data quality issues faced by ML practitioners.

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

3 | Methodology

Our study aims to discuss and evaluate issues previously introduced by harnessing a naive approach inheriting from open-source intelligence as much as possible. The sections that follow recap our key research questions and connected research approach.

3.1 | Research Questions

- RQ1: *What is the most comprehensive data quality tool for ML?*
  This question aims to evaluate and compare different data quality tools specifically designed for ML tasks. We aim to identify the tool that provides the most comprehensive set of features, functionalities, and metrics for assessing and ensuring data quality in ML workflows.
- RQ2: *What are the most relevant data quality metrics for ML?*

  This question focuses on identifying the key metrics that are crucial for ensuring data quality in the context of ML. By analyzing available tools and implemented quality checks, we aim to determine which metrics are essential for assessing the quality of data used in MLOps pipelines.
- RQ3: *What are the characteristics of Kaggle's tabular classification repositories?*

  This research question aims to investigate and describe the key metadata of tabular classification datasets available on Kaggle. By examining these characteristics, we seek to gain insights into the nature and composition of these datasets, giving a deeper understanding of the repository metadata, which are at the base of the ranking criteria (which be used to simulate user's behaviors and choose the datasets to run the quality checks on).
- RQ4: *Does Kaggle's tabular classification repositories pass data quality checks?*

  This question focuses on assessing the data quality of tabular classification datasets available on Kaggle, a popular platform for ML competitions. We aim to evaluate whether these datasets comply with the established data quality metrics and identify any potential issues or areas for improvement.

3.2 | Research Design

In this study,[2] we employed a comprehensive approach to evaluate the data quality of Kaggle datasets. Our methodology consists of four interconnected phases addressing our research questions as shown in Figure 1.

1. **Data collection and mining:** First, we interacted with the Kaggle API to retrieve classification datasets in tabular format. We then implemented a web scraping mechanism to gather additional repository information that was not

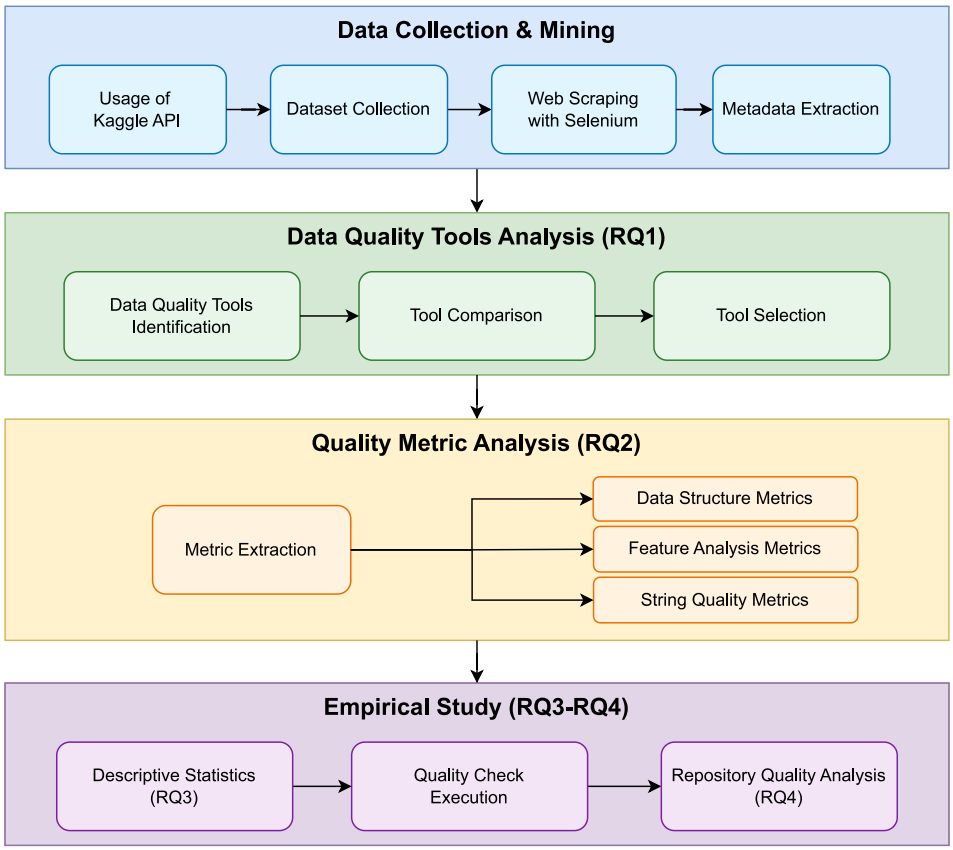

FIGURE 1 | An overview of the employed methodology.

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

accessible through the API. Finally, we extracted metadata including upvotes, usability scores, medals, and size metrics.

2. **Data quality tools analysis:** This phase aims to answer **RQ1** and begins with the identification of available data quality tools from both academic and industry sources. We evaluated the tools based on their features, maintenance status, and community support. We then selected the most appropriate tool for our empirical analysis.
3. **Quality metric analysis:** Through an analysis of the tools identified in the previous step, we examined the data quality metrics they provide. To answer **RQ2**, we categorized these metrics into three main groups: data structure metrics (examining basic quality aspects like missing values and duplicates), feature analysis metrics (focusing on correlations and outliers), and string quality metrics (analyzing textual data characteristics).
4. **Empirical study** The final phase aims to answer both **RQ3** and **RQ4**. It combines our previous findings into a comprehensive empirical analysis. We executed automated quality checks using our selected tool, focusing on the top-ranked repositories across different ranking criteria. We generated descriptive statistics about dataset characteristics and conducted detailed quality analysis, examining how different quality metrics correlate with repository rankings and popularity measures.

The data collection process was conducted through Kaggle APIs to retrieve all the CSV datasets for classification in tabular form. Because Kaggle's CSV repositories occasionally include datasets for computer vision, we wrote a script to detect the existence of other types of files and ignore them. Then we used Selenium for web-scraping to extract critical metadata of each dataset such as size, upvotes, medals, and usability evaluations. This procedure gave us an overview of the main characteristics of each dataset. We used well-known Python packages such as Pandas for analysis and Matplotlib and Seaborn for visualization to analyze and visualize the datasets' properties and attributes.

The quality selection phase involved a systematic process of identifying the most relevant data quality metrics based on their impact on ML performance. This phase prioritized key aspects such as data completeness, consistency, and integrity. It included metrics for counting missing values, feature correlations, duplicate entries, and outliers, ensuring that the data was not only clean but also representative of the real-world phenomena it aimed to model.

Next, we performed a comparative analysis of various data quality tools commonly used in the ML community by considering the selected metrics in the previous phase. We examined tools based on their ability to handle large datasets, comprehensiveness of checks, ease of integration with existing workflows, and active support from the open-source community. The result of this stage has been the selection of Deepchecks [18], for its rich set of functionalities and its ability to automate a wide range of data quality checks.

Next, we applied Deepchecks to perform the data quality evaluation on the selected datasets. The tool's comprehensive suite of checks and metrics enabled us to assess various aspects, such as data duplicates, feature correlation, missing values, mixed data types, and outlier detection, among others. We developed a customized script to download the datasets, extract them, and run all the data quality checks from the Deepchecks suite. The downloaded dataset was then immediately deleted and the suite results were saved in JSON to optimize space in the environment.

One challenge we faced while automating the checks on the downloaded datasets was the presence of pre-divided train and test datasets, as well as subfolders with different characteristics or features. To address this issue, we analyzed all possible sets associated with the datasets, including both the train and test sets. Moreover, we also considered any external CSV files with other features that may have been useful for the problem. This approach increased computational checks overload, without adding any value but we prioritized automation over analyzing each repository individually since the main train sets have been analyzed in any case. We also found a lot of duplicate rows (one per CSV file) because, as previously stated, we ran the data quality checks on all the datasets within each repository in the table of the data quality metrics. On the one hand, data de-duplication was enforced by checking duplicity in a key-resolved fashion. On the other hand, as a result, we decided to run a transformation script on this table, resulting in a new table with unique title values. For numeric features, we calculated the mean value across all duplicates. This allowed us to represent the central tendency of these numeric metrics for each duplicate. For categorical features, for which the result is either FAIL, WARN or PASS, we applied a precedence rule. If any duplicate had a result of FAIL, the overall result was marked as FAIL. If no duplicate had a FAIL but at least one had a WARN, the overall result was marked as WARN. Only if all duplicates had a result of PASS did the final result remain PASS.

The assumption remains valid because we are interested in understanding the potential data quality issues (failed checks) that were identified. To prioritize the automation aspect, the presence of some tokenization problems was simply dealt with by force-deleting these datasets.

Regarding the Outlier Sample Detection check, we computed the mean of the percentage of "outliers" over the samples in each dataset, to have a uniform metric for comparison across all the repositories. Using the mean as a metric for comparison is still valid since a higher mean value indicates a larger number of outliers detected. While this check also returns a table of potential outliers for each dataset, for our purposes, the mean provides a general understanding of this property across all the datasets. However, for more specific use cases and analysis, the table of potential outliers can be used to manually select and analyze individual outliers.

Regarding the percentage of nulls metric, we also computed the mean to create a unique metric for comparison, as the check provides a list of values for each column. It is important to note that a column with only null values still contributes to increasing this score. For single use cases, this metric is even more relevant as it allows for immediate identification of problematic features.

Our automated script is built to be both strong and versatile, and it can work with both CSV and Excel datasets. We also implemented a specific control to manage potential Unicode issues

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

with certain timestamp features by encoding the data using Latin-1. We were able to verify that our script can efficiently evaluate a wide range of datasets with diverse formats and properties by integrating these features.

By following this methodology, we aimed to provide a comprehensive evaluation of data quality in Kaggle datasets, leveraging the capabilities of Deepchecks and considering a diverse range of metrics. This approach allowed us to obtain empirical evidence and valuable insights to inform best practices in data quality assessment for ML applications.

To ensure the reproducibility of our study and to handle the large volume of data, we develop an automated script. This script is designed to download the datasets, extract them, run the data quality checks, and then delete the downloaded dataset to save space.

Through this evaluation design, we aim to provide a systematic and comprehensive assessment of data quality in ML datasets. The findings from our evaluation provide valuable insights and recommendations for practitioners in the field, promoting better data quality practices in ML applications.

4 | Empirical Evaluation

This section describes the results obtained with the analysis of selected Kaggle repositories.

4.1 | RQ1: Data Quality Software Tools

To address this RQ, we surveyed data quality tools available from academia, industry, and open-source communities. We focused on tools that primarily monitor tabular data quality, that is, tools that provide means to automate the assessment of a wide variety of data quality metrics, specifically in the context of tabular data. Notice that we excluded tools that are primarily designed for data testing (e.g., GreatExpectations[3]). These tools require users to define specific thresholds and then test whether the data meets those conditions. While this approach is valuable in many contexts, it introduces complexity and manual configuration of conditions that fall outside the scope of our study.

Below, we provide an overview of the main characteristics of the tools we analyzed. We also report relevant metadata for each tool, as retrieved from GitHub in April 2023.

- `Deepchecks` [18]: Deepchecks is the leading tool for testing, validating and monitoring your ML models and data, and it enables doing so with minimal effort. It has 1312 commits, with 41 contributors. The latest commit has been done in 04/2023.
- `DataCleaner` [19]: DataCleaner is a Data Quality toolkit that allows you to profile, correct, and enrich your data. It has 6773 commits, with 33 contributors. The latest commit has been done in 08/2022.
- `Griffin` [20]: Big Data Quality Solution For Batch and Streaming. It has 645 commits, with 53 contributors. The latest commit has been done in 09/2022.
- `Drunken Data Quality` (DDQ) [21]: DDQ is a small library for checking constraints on Spark data structures. It can be used to assure a certain data quality, especially when continuous imports happen. It has 349 commits, with 4 contributors. The latest commit has been done in 03/2018.
- `Agile Data Quality Framework` [22]: DQ is a framework to build parallel and distributed quality checks on big data environments. It can be used to calculate metrics and perform checks to assure quality on structured or unstructured data. It relies entirely on Spark. It has 52 commits, with 4 contributors. The latest commit has been done in 02/2022.
- `Talend Open Studio for Data Quality` [23]: Open source integration software. Powerful tools for you next integration project. It has 1675 commits, with 45 contributors. The latest commit has been done in 03/2023.
- `pydqc` [24]: Python automatic data quality check toolkit. It aims to relieve the pain of writing tedious codes for general data understanding by automatically generate data summary report, which contains useful statistical information for each column in a data table (useful for general data understanding). It automatically summarizes the statistical difference between two data tables (useful for comparing training set with test set, comparing the same data table from two different snapshot dates, etc.). It still need some human intervention for inferring data types inferring. It has 65 commits, with 9 contributors. The latest commit has been done in 04/2020.
- `MobyDQ` [25]: MobyDQ is a tool for data engineering teams to automate data quality checks on their data pipeline, capture data quality issues and trigger alerts in case of anomaly, regardless of the data sources they use. It has 1834 commits, with 12 contributors. The latest commit has been done in 05/2022.
- `CMS DQM` [26]: This tool is designed to automate the monitoring of data quality in large-scale experimental setups, which can be challenging to manage manually. By leveraging ML methods, the tool is trained online using manually labeled data from human experts. This allows it to continually learn and adapt to the evolving dataset. The trained model assists data quality managers by automatically filtering obvious cases of good or bad data, while only flagging ambiguous or hard-to-recognize datasets for further human evaluation. This approach optimizes the data quality monitoring process, reducing the need for constant manual review and improving the efficiency of data handling in complex experimental environments. It has 56 commits, with 5 contributors. The latest commit has been done in 07/2018.
- `Deequ` [27]: Deequ is a library built on top of Apache Spark for defining "unit tests for data", which measure data quality in large datasets. It has 212 commits, with 97 contributors. The latest commit has been done in 04/2023.

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 | Qualitative comparison among selected tools.

Name	Data duplicates	Feature correlation	Outliers detection	String length out of bounds	String mismatch
Deepchecks	✓	✓	✓	✓	✓
DataCleaner	✗	✗	✗	✓	✓
Griffin	✓	✗	✗	✗	✗
DDQ	✓	✗	✗	✓	✓
DQ	✓	✗	✗	✗	✗
Talend	✓	✗	✗	✓	✓
pydqc	✓	✗	✗	✓	✓
MobyDQ	✓	✗	✗	✓	✓
CMS DQM	✓	✓	✗	✗	✗
Deequ	✓	✗	✗	✓	✓

As we can see from this recap, many of these tools are no longer completely supported, with several having not been updated for multiple years. However, this does not necessarily imply that their functionalities are no longer usable or relevant. For example, tools like DDQ and CMS DQM, despite their last commits being in 2018, could be useful in specific scenarios, particularly when working with large-scale experimental data. In contrast, other tools like Deepchecks, Deequ, and Talend Open Studio for Data Quality have more recent commits, indicating continued support and updates. The relevance of each tool should be considered in light of both its current functionality and the needs of the specific use case.

In this analysis, we used a comparison approach, to better underline the key differences between the tools. Table 1 shows the results of evaluating the chosen metrics, with ✓ representing the possibility of running that check. On the contrary the table reports ✗ if the check cannot be carried out.

For the sake of readability, Table 1 omits certain metrics that are implemented by all the tools reviewed. These metrics include missing values, mixed data types, mixed nulls, single value, and percent of nulls.

Additionally, it is worth highlighting that Deepchecks stands out as one of the most up-to-date, comprehensive, and rapidly evolving tools in this domain. Beyond its robust feature set, it distinguishes itself through its simplicity and ease of use, making it particularly well-suited for beginners. This combination of accessibility and functionality underscores its value and relevance for ML practitioners.

4.2 | RQ2: Data Quality Metrics

As anticipated above, our focus is on tabular data, and we identified a subset of metrics particularly relevant to ML pipelines. We elicited existing metrics through a systematic analysis of available tools, focusing on their documentation, user manuals, and outputs derived from tool validation activities. Specifically, we defined the scope of our analysis to focus on data quality in ML during pre-production phases, with an emphasis on tabular datasets and classification tasks.

To conduct this analysis, we started from the representative set of tools for data quality assessment in ML identified in the context of RQ1. We reviewed their documentation and manuals to extract details about supported metrics and conducted validation activities to confirm the practical applicability of these metrics. This approach allowed us to systematically catalog metrics tailored to the needs of ML pipelines, ensuring relevance to classification tasks and pre-production phases.

The final set of metrics used for tool comparison is outlined below. Each metric is accompanied by a brief description and references to the sources that discuss it.

- *Data duplicates*: This check detects duplicate samples in the dataset. Duplicates can affect the model's training by increasing the weight given to those samples, potentially leading to overfitting and poor generalization [28]. While intentional oversampling or identical-looking samples may be valid reasons for duplicate samples, unexpected duplicates may indicate problems in the data pipeline that need to be addressed. Therefore, identifying and handling data duplicates is an important step in ensuring data quality for ML applications.
- *Feature-feature correlation*: This check computes pairwise correlations between features, potentially identifying highly correlated pairs of features, which can be deleted since they can lead to skewed or misleading results [29].
- *Missing values*: This check calculates the percentage of null values for each column. In general, this issue can lead to loss of accuracy of the ML model or even bias in the results of the predictions [30].
- *Mixed data types*: This check looks for columns containing a mix of string and numeric values and identifies them as well as their respective ratios. Mixed data types in a single

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

column can lead to errors in ML models and reduce their performance. Furthermore, certain ML algorithms are designed to function primarily with numeric data, and attempting to fit these models with mixed data types can cause problems. Identifying columns with mixed data types is therefore critical for guaranteeing data quality and preparing data for ML activities [31].

- *Mixed nulls*: This check looks for different types of null values, such as string representations of null. In order to handle missing data correctly, it is critical to determine all alternative representations of null values. Failure to do so can result in erroneous results and unreliable models. The occurrence of mixed nulls in a dataset might also suggest problems with data collection or processing, which should be rectified before using the data for modeling [32].
- *Outlier sample detection*: A data point that stands out from the others is called an outlier. They highlight measurement mistakes, poor data collection, or simply show variables that were not taken into account during data gathering. It is important to note that while some ML models may perform well despite the presence of outliers, others can fail completely, depending on their design and how they are built [33].
- *Single value in column*: This check identifies columns that have only a single unique value in all rows. Such columns are known as constant features and do not provide any value to ML models. It is recommended to exclude constant features from ML models, as they do not contribute to model performance and may even cause overfitting. Regularization methods, which penalize large coefficients in the model, often exclude constant features as unimportant [34].
- *Special characters*: This check helps identify the presence of special characters in the dataset. This check is especially important for preprocessing natural language processing (NLP) tasks, as certain characters, such as "/", "==", and "?", do not add value to the text [35].
- *String length out of bounds*: This check finds strings that are much longer or shorter than the expected "normal" string lengths. It can assist in identifying outliers in the dataset as well as potential problems with data entry or collection.
- *String mismatch*: This check detects mismatches in each string column in the data, which can be helpful for identifying errors in the data. For example, if your data is aggregated from multiple sources, it may have the same values but with slight variations in formatting, such as a leading uppercase letter. In this case, the model's ability to learn may be impaired since it sees categories that are supposed to be the same as different categories. By identifying and addressing these mismatches, the data can

TABLE 2 | List of data quality metrics (data structure).

Metric	Formula	Objective
Duplicates	`duplicate_percentage` $= 1-(1.0 \cdot \texttt{int(n_unique)})/(1.0 \cdot \texttt{int(n_samples)})$	Minimize
	• `n_unique` = number of datasets' columns with just unique values	
	• `n_samples` = number of samples (rows) of the dataset	
Missing values	`data[col].isna().sum()`, $\forall$ `col` $\in$ `dataset_features`	Minimize
	• `data` = sample of the dataset	
	• `col` = single feature of the dataset	
	• `dataset_features` = all the features of the dataset	
Mixed data types	Numerical mix for each column	Minimize
	`nums_pct = nums/total_rows`	
	• `nums` = count of numeric values	
	• `total_rows` = total number of non-null values	
Mixed nulls		Minimize
	`mixed_nulls[%columns%][column_name][null_value]` = {*count*, *percent*}	
	• `mixed_nulls[%columns%]` = dictionary that holds information about each columns' nulls	
	• {*count*, *percent*} = count (number of occurrences of the null value in the column) and percent (proportion of null values in the column as a percentage of the total number of values in that column)	
Single value in column	`num_unique_per_col = data.nunique(dropna=self.ignore_nan)`	Delete if holds
	• `num_unique_per_col` = number of unique values per column	

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

be cleaned and prepared for better performance in ML models.

It is important to note that the list above is not exhaustive. From the whole set of metrics identified in our analysis, we intentionally excluded those that require domain knowledge. As an example, we exclude class imbalance [36] and feature-label correlation [37], as they require domain-specific knowledge, like label classes, which cannot be automatically derived. Labels can vary in both naming and positioning across different datasets, complicating the automation of these calculations. We also excluded all metrics for evaluating runtime phenomena, such as Concept Drift [38], as our primary focus is on assessing and ensuring data quality during pre-production phases.

We provide further details for all collected metric including definition, formula, corresponding description, and objectives (whether to be minimized, maximized, or eliminated). The metrics are categorized by type: data structure in Table 2, feature analysis in Table 3, and string quality in Table 4.

4.3 | RQ3: Mined Data Collection Metadata Analysis

Here we analyze the Kaggle data mined and their metadata. In particular, we analyzed all the classification repositories gathered by scraping Kaggle web pages. Kaggle characterizes its datasets based on various factors. Specifically, we extracted Kaggle's main repository metadata, which includes:

- Upvotes: community votes (from 0 to ∞).
- Usability: a score that rates how easy-to-use a dataset is based on factors like documentation, related public content, file types, and metadata coverage (from 0 to 10).

TABLE 3 | List of data quality metrics (feature analysis).

Metric	Formula	Objective
Feature-feature correlation	Given X and Y random variables • Numerical-numerical correlations: Spearman's rank correlation coefficient $r_s = \frac{Cov(X)Cov(Y)}{\sigma_X \sigma_Y}$ • Categorical-categorical correlations: Theil's U coefficient $U(X,\ Y) = \frac{H(X)U(X\|Y)+H(Y)U(Y\|X)}{H(X)+H(Y)}$	Delete if above threshold
Outlier sample detection	Formula of LoOP[39]: $LoOP_s(o):\ = \max\left\{0, erf\left(\frac{PLOF_{\lambda,S}(o)}{nPLOF * \sqrt{2}}\right)\right\}$ • LoOP = Local outlier probability of data 'o' • PLOF = ratio of the estimation for the density around data 'o' and the expected value of the estimations for the densities around all objects in the context set S(o)	Delete
Special characters	`special_samples = _get_special_samples(column_data)` • `special_samples` = special characters in one column of the dataset • `column_data` = feature of the dataset • `_get_special_samples` = function that counts and returns the samples of a column that are string but not in the base form	Delete
String length out of bounds	`outliers = str_length_column.between(lower,upper,inclusive = %both%)` • `str_length_column` = length of nearest quantile calculated concerning the strings of a column • `lower,upper` = minimum and maximum length range	Delete
String mismatch	`mismatch_strings[~columns~][base_form].append( variant, count, percent )` • `base_form` = base form of strings from collections library • `variant` = finds variant structures • `count` = count the mismatches	Delete

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 4 | List of data quality metrics (string quality).

Metric	Formula	Objective
Special characters	`special_samples = _get_special_samples(column_data)` • `special_samples` = special characters in one column of the dataset • `column_data` = feature of the dataset • `_get_special_samples` = function that counts and returns the samples of a column that are string but not in the base form	Delete
String length out of bounds	`outliers = str_length_column. between(lower, upper, inclusive=~both~)` • `str_length_column` = length of nearest quantile calculated concerning the strings of a column • `lower,upper` = minimum and maximum length range	Delete
String mismatch	`mismatch_strings[~columns~][base_form]. append( variant, count, percent )` • `base_form` = base form of strings from collections library • `variant` = finds variant structures • `count` = count the mismatches	Delete

- Medal: achievement based on the number of votes and represents achievements for the user who created the dataset. A repository could have no medal (encoded as 0), or either a bronze (encoded as 1), silver (encoded as 2), or gold (encoded as 3) medal.
- Size: the dimension of the dataset in megabytes (MB) (from 0 to ∞).

To gain a more comprehensive understanding of the characteristics of the repositories, we conducted a deeper analysis of their metadata as follows.

Considering the Upvotes in Figure 2, which determines the Most votes ranking, we can see that the actual cumulative curve is highly compressed around low values. Specifically, the function has a mean of 36 upvotes, and 75% of the values are below 28 upvotes. The maximum number of votes is 3049. We may notice significantly better values when we evaluate the Usability metadata as shown in Figure 3. The mean value is around 7.7 with more than 50% of values greater than 8.2. The Usability plot emphasizes the importance of documentation, explainability of datasets, and metadata in general for dataset curators. If we consider the distribution of medals in Figure 4, encoded as gold (3), silver (2), bronze (1), and null (0), we can see that more than the 56% of the medals are gold, while the 36% have no medal.

Finally, we analyzed the size of the dataset, plotted in Figure 5 (x-axis in 10^7 bytes). This feature is not related to any ranking feature but yields the dimension of datasets practitioners commonly employ. The mean of the distribution is around 900 MB but the 75% of datasets are smaller than 30 MB.

This confirms that most of the tools previously examined would be ineffective for this purpose as their approach and data quality

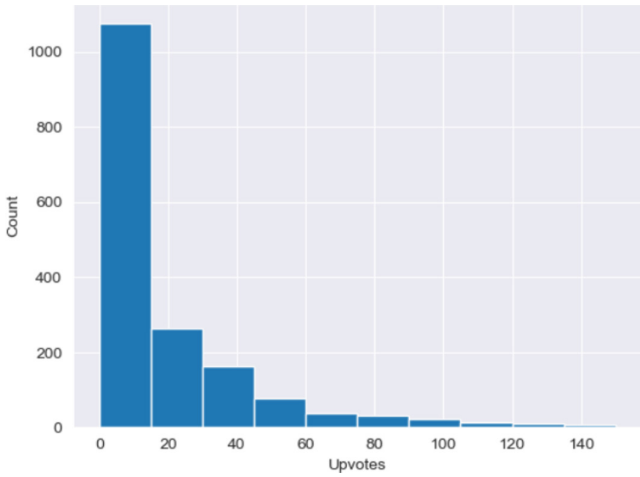

FIGURE 2 | Distribution of upvotes.

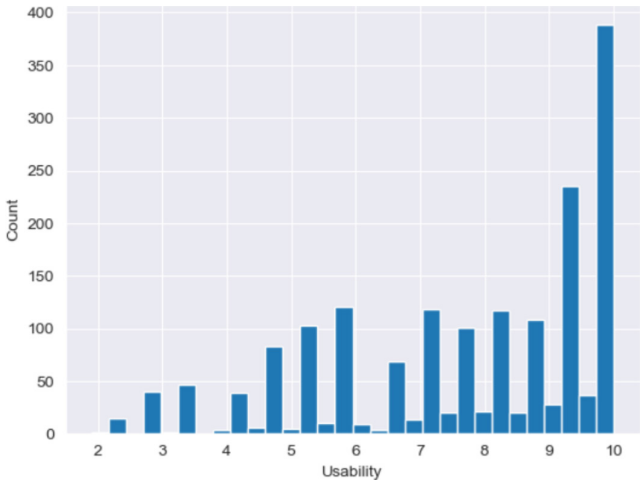

FIGURE 3 | Distribution of usability.

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

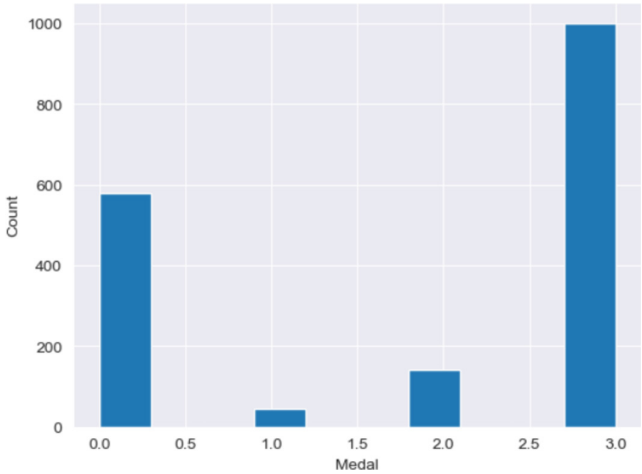

FIGURE 4 | Distribution of medals.

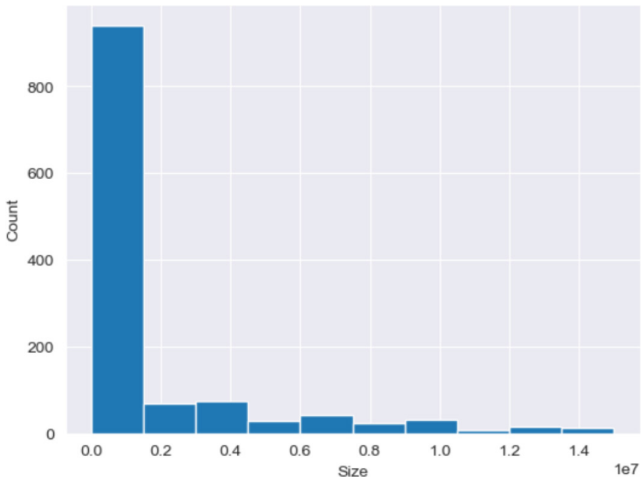

FIGURE 5 | Distribution of sizes.

evaluations are tailored for (e.g., medium-sized) datasets yet conversely most if not all frequently utilized datasets are smaller and more manageable than what assumed. For this reason and for computational efficiency in downloading and processing more than 1000 datasets we decided to add the constraint of considering just datasets of size lower than 500 MB, which is still relevant for our analysis, discarding other possible larger datasets.

4.4 | RQ4: Data Quality Checks Evaluation

When a user wants to work on a Kaggle dataset, the ranking criteria that determine which datasets are shown to the community are mostly self-explanatory and they are:

- Hotness: ranking based on a pull of different metadata such as upvotes, usage, and usability.[4]
- Most votes: ranking based on the number of upvotes.
- New: ranking based on latest uploaded repositories.
- Updated: ranking based on latest updated repositories.
- Usability: ranking based on usability metadata.

Note that in Kaggle, *hotness* and *upvotes* are two distinct metrics used to evaluate and rank datasets. Metric upvotes represents a measure of long-term popularity, where users can upvote datasets they find useful or high-quality. On the other hand, metric hotness is a dynamic and captures the recent popularity or engagement with a dataset, based on factors such as downloads, comments, and interactions over a shorter, more recent period. Unlike upvotes, the hotness ranking fluctuates based on recent activity, making it a useful indicator of which datasets are currently attracting attention, even if they have not yet accumulated many upvotes. All of these ranking criteria are directly related to the metadata we chose, making our experiment highly relevant and whose numerical validity followed established rigorous software metric approaches [41]. Our goal was to explore the relationship between Kaggle's ranking criteria and potential data integrity issues in the datasets. To simulate user behavior, we made the assumption that most users would not browse beyond the second or third page of repositories when selecting a dataset to work with. In this context, "pages" refers to the paginated results on Kaggle's platform, where each page lists a limited number of repositories sorted by various metrics such as popularity or relevance. To ensure a comprehensive analysis, we decided to examine Kaggle's sorting criteria and perform data quality checks on the first 100 datasets—equivalent to browsing through the first five pages of results.

The three rankings analyzed are based on: Hotness, Most votes, and Usability metrics since they represent the most used ranking features. Table 5 shows the descriptive statistics for the datasets, whereas Table 6presents the results of quality checks. In the tables, for each analyzed metric we present the mean (μ), the 75th percentile ($p75$), and the maximum value for numerical features, whereas we show the percentage of failed tests (Fail), the percentage of tests that received warning (Warn), and the percentage of passed checks (Pass) for categorical metrics.

Concerning Hotness Ranking, datasets tend to be relatively small in size, with a 75th percentile of 2 MB. This aligns with Kaggle's intent to create an accessible starting point for newcomers, as smaller datasets are less intimidating and easier to work with. These datasets typically have high usability scores and a substantial number of votes, further enhancing their appeal. The prevalence of missing values and outliers is low, allowing beginners to focus on fundamental problems without being overwhelmed by complex data quality issues.

However, certain limitations persist. Approximately 30% of datasets in this category trigger warnings about data duplicates, and 40% exhibit high feature-to-feature correlation, which can complicate modeling efforts. About 12% of the datasets contain single-value columns, which are not informative for ML models, highlighting areas where data curation could improve.

Considering Most votes ranking, datasets have a smaller mean but higher 75th percentile and maximum values and similar percentages of missing values compared with those in the Hotness category. This may suggest that these datasets are more complex and appealing to experienced practitioners who are comfortable

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 5 | Descriptive statistics by metric and dataset type.

	μ			p75			Max		
Metric	**Hotness**	**Most votes**	**Usability**	**Hotness**	**Most votes**	**Usability**	**Hotness**	**Most votes**	**Usability**
Upvotes	243	291	57	241	255	64	3049	3049	417
Usability	9.3	9.2	10	10	10	10	10	10	10
Size	134	39	32	2	4	6	708	945	945
Medals	Bronze	Bronze	Bronze	Silver	Silver	Silver	Gold	Silver	Gold

working with more challenging data. With greater complexity comes a need for caution, as poorly managed datasets can propagate errors into models. Datasets in this category show higher feature correlation and an increased percentage of single-value columns compared with the Hotness category. This suggests that while the datasets may be larger, they do not necessarily offer more informative features, providing an opportunity for users to refine their preprocessing skills. Despite these challenges, the proportion of missing values and outliers remains relatively small.

Concerning Usability ranking, datasets exhibit similar characteristics with the Hotness and Most votes categories. However, this ranking also demonstrate higher feature correlation and data duplicates. Additionally, warnings related to string quality issues are common, indicating areas where data preprocessing can be improved.

Across all rankings, datasets present varying levels of quality issues, with feature correlation, duplicates, and single-value columns being recurring concerns. While datasets in the Hotness category are ideal for beginners, the larger and more complex datasets in the Most votes and Usability rankings cater to more experienced users. However, these datasets still pose challenges that require careful preprocessing to ensure reliable model performance.

Our results show that even the highest-rated and top-ranked datasets often exhibit data integrity issues. These issues can have a significant impact, introducing errors and biases into models, especially if left unchecked. Such problems may remain unnoticed by practitioners, leading to unintended and uncontrolled consequences that compromise the reliability of ML systems.

Despite the critical role of data quality in ML pipelines, evidence shows that many practitioners neglect to perform comprehensive data quality checks. This underscores the need for greater awareness and education about the importance of ensuring data integrity as a foundational step in building robust models.

4.5 | Observations and Lessons Learned

Our study shows that open-source datasets—while having become indispensable in advancing ML research and applications—exhibit low-quality but high accessibility, which enables practitioners to test algorithms, benchmark models, and prototype solutions efficiently but with unfathomable longer-term consequences. Data quality issues, biases, and limitations inherent to such datasets may indeed compromise the reliability of ML systems.

Based on our findings, we recommend the following design principles for practitioners to follow when working with Kaggle datasets or any other data sources for ML:

- *Carry out rigorous data quality checks*: A rigorous data quality evaluation framework that investigates data duplicates, feature correlations, heterogeneous data types, inconsistent null values, columns with singular values, special characters, and string discrepancies ensure the detection and resolution of potential data integrity issues.
- *Endorse the use of homogeneous and standardized labeling conventions across datasets*: Endorsing the use of homogeneous and standardized labeling conventions across datasets help automated data quality assessments. This is especially beneficial for measurements like class imbalance and feature-label correlations.
- *Prioritize robust documentation and metadata*: The provision of detailed documentation and metadata for datasets, inclusive of explicit feature descriptions, anticipated data types, and potential data quality issues is critical. This empowers practitioners to make well-informed decisions during dataset selection and preprocessing stages.
- *Encourage collaboration and knowledge sharing*: Creating a conducive environment for collaboration and knowledge exchange among practitioners through dedicated platforms or forums fosters collective learning. This enables practitioners to leverage shared experiences and insights related to data quality.

In summary, our research unveils the prevalent occurrence of data integrity issues in Kaggle datasets, stressing the necessity for meticulous data quality checks within ML workflows. By integrating comprehensive data quality evaluations and complying with our suggested design principles, practitioners can enhance the reliability, precision, and generalizability of their ML models. This, in the long run, drive superior outcomes and decision-making processes.

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 6 | Quality checks by metric and dataset type.

Metric	μ			p75			Max (%)		
	Hotness	Most votes	Usability	Hotness	Most votes	Usability	Hotness	Most votes	Usability
Missing Values	0.023	0.028	0.027	0.00068	0.015	0.016	0.67	0.29	0.49
Outliers Detection	0.11	0.12	0.094	0.16	0.15	0.14	0.21	0.21	0.22
	Fail (%)			Warn (%)			Pass (%)		
	Hotness	Most votes	Usability	Hotness	Most votes	Usability	Hotness	Most votes	Usability
Data duplicates	0	0	0	29	29	42	71	71	58
Feature correlation	40	51	48	0	0	0	60	49	52
Mixed data types	0	0	0	6	7	7	94	93	93
Mixed nulls	5	9	9	0	0	0	95	91	91
Single value in column	12	22	26	0	0	0	88	78	74
Special characters	0	0	0	10	9	11	90	91	89
String length out bounds	0	0	0	15	20	29	85	80	71
String mismatch	0	0	0	33	29	39	77	71	61

In the following we discuss the problems implied by our RQs and their respective results, especially rotating around open-source datasets and their potential implications for ML applications. The reflections were obtained by presenting said results to a panel of three experts (10+ years of experience) MLOps practitioners in the scope of a 2-h panel to help us synthesize concrete lessons learned.

4.5.1 | Incomplete and Missing Data

Many open-source datasets suffer from missing or incomplete data. Attributes may be partially filled, or entire instances may lack critical features. Such issues are often the result of inadequate data collection protocols or human error during dataset compilation. When used without preprocessing, these gaps can lead to biased models or reduced performance.

4.5.2 | Noisy Data

Open-source datasets frequently include noisy data, such as mislabeled samples, incorrect values, or duplicate records. Noisy labels can significantly degrade the training process of supervised models, leading to incorrect generalizations and predictions. This is particularly problematic in domains like healthcare or autonomous driving, where errors have critical consequences.

4.5.3 | Outdated Data

Datasets often do not reflect the current state of the environment or population they aim to represent. For instance, social media datasets may contain slang, terminology, or behaviors that are no longer relevant. Using outdated datasets risks training models that fail to perform effectively on current data distributions.

4.5.4 | Sampling Bias

Open-source datasets are frequently curated only under specific conditions or constraints that introduce sampling bias. For instance, datasets collected from urban regions may not generalize well to rural areas, leading to underrepresented populations or use cases. Models trained on such data perpetuate inequities or perform poorly in unseen scenarios.

4.5.5 | Cultural and Demographic Bias

Datasets may overrepresent certain demographics while neglecting others. For example, facial recognition datasets often overrepresent light-skinned individuals, resulting in reduced accuracy for darker-skinned individuals. Such biases can exacerbate societal inequities and lead to ethical concerns.

4.5.6 | Intentional Manipulations and Adversarial Biases

Open-source datasets may contain adversarially manipulated data, particularly in competitive ML contexts. Injecting subtle

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

corruptions or poisoning data can affect the integrity of downstream tasks, emphasizing the need for rigorous data vetting.

4.5.7 | Licensing Issues

Some open-source datasets have unclear licensing terms, potentially restricting their use in commercial or sensitive applications. Using such data without proper validation may lead to legal disputes or unintended violations.

4.5.8 | Privacy Concerns

Datasets that include personally identifiable information (PII) or sensitive attributes may not adhere to data protection regulations such as General Data Protection Regulation or California Consumer Privacy Act. Even when anonymized, datasets might still allow for re-identification of individuals, posing privacy risks.

4.5.9 | Domain Mismatch

Datasets designed for one domain do not transfer well to another. For instance, a sentiment analysis dataset based on movie reviews may not work well in analyzing legal documents due to domain-specific vocabulary differences.

4.5.10 | Lack of Scalability

Many open-source datasets are relatively small and fail to capture the scale of data encountered in real-world applications. Models trained on such datasets may overfit or fail to generalize in production.

4.6 | Threats to Validity

The empirical nature of our study, combined with the inherent limitations of research at this scope, introduces several threats to validity that warrant further investigation in future work.

First, we excluded several valuable metrics, such as Class Imbalance, as they require precise knowledge of label classes for each dataset—information that cannot be obtained through an automated procedure. This decision reflects the practical reality where labels are often inaccessible without extensive manual intervention. However, this choice also limits the extent to which our study inherits from the state-of-the-art in software engineering practices. Future work should focus on developing automated methods to detect these so-called MLOps smells, offering a clearer and more comprehensive overview of such issues.

Another limitation related to the timing and process of data integrity checks. The most effective time to run these checks is before model training, following raw data preprocessing. However, this step is highly dependent on user-specific workflows and the unique configurations of MLOps pipelines, which vary significantly. Analyzing each pipeline component in detail—particularly tools like Jupyter notebooks, which often serve as the foundation of many MLOps workflows—would be essential for a more comprehensive approach.

The comparative analysis performed in this study is relatively simple and focuses on identifying potential issues in the highest-ranked projects. It does not attempt to establish correlations between ranking categories (e.g., hotness, most votes, and usability) and specific types of issues (e.g., data structure, feature analysis, and string quality). Investigating whether such correlations exist with statistical significance is beyond the scope of this work. Future research will aim to address this gap by conducting more rigorous experiments to explore these relationships. Additionally, we plan to replicate the study using diverse data sources to enhance external validity and strengthen the generalizability of the findings.

5 | Conclusion

In this study, we explore data quality tools and show the capabilities of Deepchecks for ML applications. Specifically, we employ the data integrity suite of Deepchecks to analyze multiple tabular datasets from Kaggle, focusing on a subset of metrics particularly relevant to ML pipelines, especially in the context of classification tasks.

Our findings highlight the importance of data quality checks before training ML models. Many datasets commonly used by practitioners are not as clean as assumed, leading to potential quality issues in model issues that practitioners may remain unaware of. Additionally, Kaggle's ranking criteria are not always aligned with data quality and integrity. Several datasets that fail to meet our quality standards have nonetheless received high rankings based on factors such as upvotes and medals. This misalignment highlights the necessity of rigorous data preprocessing and quality assurance checks to avoid unreliable models.

While some limitations remain, such as domain-specific constraints and the scope of quality checks, our research demonstrates the significant potential of automated tools like Deepchecks in streamlining and enhancing dataset quality assessments. Notice that our research does not establish statistically significant correlations between ranking categories and specific types of issues. Future research will aim to address this gap by conducting additional experiments in this direction.

We plan to extend this study to computer vision tasks, focusing on the unique challenges and evaluation metrics inherent to these problems, which differ significantly from those encountered with tabular data. Computer vision applications are gaining widespread popularity. Additionally, we aim to examine Kaggle's updated ranking criteria to assess whether the latest repositories demonstrate improved statistical performance compared with the most commonly used ones. Another promising direction for future research involves analyzing the notebooks associated with each repository. This analysis will investigate whether the most-voted or highest-ranked notebooks effectively address initial data quality issues or if these problems persist despite their popularity or ranking.

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Acknowledgements

Open access publishing facilitated by Politecnico di Milano, as part of the Wiley - CRUI-CARE agreement.

Data Availability Statement

The data that support the findings of this study are openly available in haggle-study at https://github.com/deib-polimi/kaggle-study.

Endnotes

[1] https://www.kaggle.com/.

[2] The replication package of this study is available at https://github.com/deib-polimi/kaggle-study.

[3] https://github.com/great-expectations/great_expectations.

[4] Note that Hotness ranking metric is not public available, publicly available to prevent people from exploiting the system [40] so it is not possible to link it explicitly to our features.

References

1. T. M. Mitchell, *Machine Learning* (McGraw-Hill, 2010).

2. D. P. Youll, *Software Engineering: Principles and Practice*, ed. H. van Vliet (Wiley, 1993), 558 pp 19.95. ISBN 0 471 93611 1.. Inf. Softw. Technol. 1994; 36(1): 58.

3. D. A. Tamburri, *Sustainable MLOps: Trends and Challenges* (IEEE, 2020), 17–23.

4. I. Kumara, F. Pecorelli, G. Catolino, R. Kazman, D. A. Tamburri, and W. J. Van Den Heuvel, *Architecting MLOps in the Cloud: From Theory to Practice* (IEEE, 2023), 333–335.

5. A. Jain, H. Patel, L. Nagalapatti, et al. "Overview and Importance of Data Quality for Machine Learning Tasks". In: KDD'20. Association for Computing Machinery; New York, NY, USA: 35613562, (2020)

6. L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*, 1st ed. (Addison-Wesley Professional, 2015).

7. A. Serebrenik, W. Poncin, and Brand v. dM., "Process Mining Software Repositories: Do Developers Work as Expected?" *ERCIM News* 2012, no. 88 (2012): 16–17.

8. J. M. Zhang, M. Harman, L. Ma, and Y. Liu, "Machine Learning Testing: Survey, Landscapes and Horizons," *IEEE Transactions on Software Engineering* 48, no. 1 (2022): 1–36, https://doi.org/10.1109/TSE.2019.2962027.

9. N. Hynes, D. Sculley, and M. Terry, "The Data Linter: Lightweight Automated Sanity Checking for ML Data Sets". (2017).

10. E. Breck, N. Polyzotis, S. Roy, S. Whang, and M. Zinkevich, "Data Validation for Machine Learning," in *Proceedings of the Second Conference on Machine Learning and Systems, SysML 2019, Stanford, CA, USA*, eds. A. Talwalkar, V. Smith, and M. Zaharia (2019), 2019mlsys.org.

11. D. Baylor, E. Breck, H. T. Cheng, et al. "TFX: A TensorFlow-Based Production-Scale Machine Learning Platform". In: KDD'17. Association for Computing Machinery; New York, NY, USA: 13871395, (2017)

12. J. Vanschoren, J. N. Van Rijn, B. Bischl, and L. Torgo, "OpenML: Networked Science in Machine Learning," *ACM SIGKDD Explorations Newsletter* 15 (2014): 49–60 abs/1407.7722.

13. H. Mohanty, P. Bhuyan, and D. Chenthati, eds., *Big Data: A Primer. 11 of Studies in Big Data* (Springer, 2015).

14. S. R. Kaminwar, J. Goschenhofer, J. Thomas, I. Thon, and B. Bischl, "Structured Verification of Machine Learning Models in Industrial Settings," *Big Data* 11, no. 3 (2023): 181–198, https://doi.org/10.1089/big.2021.0112.

15. M. A. de F. Farias, R. Novais, M. C. Júnior, L. P. de Silva Carvalho, M. Mendonça, and R. O. Spínola, "A Systematic Mapping Study on Mining Software Repositories," in *SAC'16* (New York, NY, USA: Association for Computing Machinery, 2016), 1472–1479.

16. M. F. Bosu and S. G. MacDonell, *A Taxonomy of Data Quality Challenges in Empirical Software Engineering* (IEEE, 2013), 97–106.

17. X. Wu, W. Zheng, X. Xia, and D. Lo, "Data Quality Matters: A Case Study on Data Label Correctness for Security Bug Report Prediction," *IEEE Transactions on Software Engineering* 48, no. 7 (2022): 2541–2556, https://doi.org/10.1109/TSE.2021.3063727.

18. Deepchecks. Code Available at:. https://github.com/deepchecks/deepchecks;.

19. DataCleaner. Code Available at:. https://github.com/datacleaner/DataCleaner;.

20. Griffin. Code Available at:. https://github.com/apache/griffin;.

21. FRosner. Code Available at:. https://github.com/FRosner/drunken-data-quality;.

22. dev a.-l. Code Available at:. https://github.com/agile-lab-dev/DataQuality;.

23. Talend. Code Available at:. https://github.com/Talend/data-quality;.

24. SauceCat. Code Available at:. https://github.com/SauceCat/pydqc;.

25. mobydq. Code Available at:. https://github.com/mobydq/mobydq;.

26. yandexdataschool. Code Available at:. https://github.com/yandexdataschool/cms-dqm;.

27. awslabs. Code Available at:. https://github.com/awslabs/deequ;.

28. AnmolNarang. How Do Duplicates Affect ML Model?. https://www.kaggle.com/questions-and-answers/200598; 2021.

29. Badr W. Why Feature Correlation Matters. A Lot! https://towardsdatascience.com/why-feature-correlation-matters-a-lot-84 7e8ba439c4; 2019.

30. T. Nawale, "How to Deal With Missing Values in Machine Learning". https://medium.com/geekculture/how-to-deal-with-missing-values-in-machine-learning-98e47f025b9c; (2022).

31. S. Eade, "Data Cleaning With Pandas Avoid This Mistake!". https://towardsdatascience.com/data-cleaning-with-pandas-avoid-this-mistake-7af559657c2c; 2020.

32. S. Kumar, "7 Ways to Handle Missing Values in Machine Learning". https://towardsdatascience.com/7-ways-to-handle-missing-values-in-machine-learning-1a6326adf79e; (2020).

33. M. Inuwa, "Outliers and Overfitting When Machine Learning Models Cant Reason". https://www.analyticsvidhya.com/blog/2022/07/outliers-and-overfitting-when-machine-learning-models-cant-reason; (2022).

34. T. Jegan, "Why to Remove Columns With 1 Unique Value While Data Cleaning Process?". https://www.kaggle.com/questions-and-answers/269943; (2021).

35. Shilpijs. "Text Preprocessing in Python - Getting Started With NLP". https://www.analyticsvidhya.com/blog/2021/08/text-preprocessing-in-python-getting-started-with-nlp/; (2021).

36. C. X. Ling and V. S. Sheng, *Class Imbalance Problem*, ed. 171 (Boston, MA: Springer US, 2010), 171.

37. Z. Zhang, Z. Zhang, J. Yao, et al., "Multi-Label Feature Selection Via Adaptive Label Correlation Estimation," *ACM Transactions on*

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Knowledge Discovery from Data 17, no. 9 (2023), https://doi.org/10.1145/3604560.

38. F. K. Khattak, V. Subasri, A. Krishnan, et al. "MLHOps: Machine Learning for Healthcare Operations" (2023).

39. H. P. Kriegel, P. Kröger, E. Schubert, A. Zimek, "LoOP: Local Outlier Probabilities". In: CIKM'09. Association for Computing Machinery; 2009; New York, NY, USA: 16491652

40. M. O'Neill, "Hotness Calculation Formula". https://www.kaggle.com/general/39290;.

41. N. E. Fenton and S. L. Pfleeger, *Software Metrics: A Rigorous and Practical Approach*, 2nd ed. (PWS Publishing Co., 1998).

20477481, 2025, 9, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70044, Wiley Online Library on [16/02/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License